TORRENTNG / ENGINE REBUILD

Code audit and rebuild blueprint

A pinned-source assessment of transfer correctness, I/O ownership, memory, CPU, scheduling, and the evidence required to earn a performance claim.

35	63	28	30
code findings	work packages	workload families	focused tests pass

The test count applies to the narrow v2 extent correction. The independent request-window regression remains open. The complete engine is specified for rebuild, not represented as already rewritten.

Audited repository

github.com/snapetech-org/TorrentNG

`05d5c23886446f380552c73e0be480f03ae2d2ab`

17 October 2026

Includes source links, measured evidence, architecture, gates, and execution order.

Guide to this report

Start with the audit for the defects and their evidence. The architecture specifies ownership, migration and release gates. The qualification contract defines fair measurements. The full companion ZIP includes 63 scoped tickets, detailed transfer/storage specifications, executable regressions, raw logs, and strict result schemas.

Guide to this report 2

Part I - Code audit and evidence 4

1. Verdict . . . 4
2. What was verified . . . 4
3. Transfer and protocol findings . . . 6
4. Memory, CPU, and storage findings . . . 9
5. Control plane, tests, and evidence findings . . . 12
6. What the checked-in performance results actually establish . . . 15
7. What to retain, replace, and defer . . . 16

Part II - Replacement architecture 17

1. Product contract . . . 17
2. Ownership boundaries . . . 17
3. The data path . . . 18
4. Capacity follows the data . . . 19
5. Scheduling policies . . . 20
6. Storage and durability details . . . 21
7. Control plane and ancillary I/O . . . 21
8. Migration without a dangerous rewrite cliff . . . 22
9. Dependency-ordered delivery gates . . . 23
10. First implementation sequence . . . 23
11. Effort and decision discipline . . . 24

Part III - Benchmark and release contract 25

1. What this contract measures . . . 25
2. Evidence motivating the suite . . . 25
3. Controller structure and adapter contract . . . 26
4. Build and baseline identity . . . 26
5. State machine, timestamps, and denominators . . . 26
6. Resource accounting . . . 27
7. Filesystem, cache, and hardware controls . . . 28
8. Independent ceilings and protocol controls . . . 28
9. Trials, statistics, and invalidity . . . 29
10. Proposed gates . . . 29
11. Qualification and rollout . . . 30
Primary references . . . 30

Part IV - Proposed acceptance scorecard **31**

Conditions for a qualifying result . . . 31
Initial gates . . . 31
Comparative claims . . . 32

Part V - Implementation work index **33**

Execution frontiers . . . 36

Source index **37**

Primary technical references . . . 39

Part I - Code audit and evidence

Audited source: `snapetech-org/TorrentNG`, commit `05d5c23886446f380552c73e0be480f03ae2d2ab`. **Audit date:** 17 October 2026. All code links in this report are pinned to that revision. The original checkout was left unchanged; regression tests and a narrow proof fix were exercised in an isolated worktree.

1. Verdict

TorrentNG has fundamental transfer-path defects and an evidence problem. It contains serious work on filesystem safety, bounded queues, crash recovery, API compatibility, and storage primitives. Those strengths do not compose into an efficient transfer engine. Work is repeatedly copied, serialized, rescanned, or retried at the boundaries between otherwise reasonable components. Some performance-oriented abstractions are bypassed by the live path; some accounting describes a projection rather than the memory it purports to bound.

The sharpest finding is a correctness failure, not a tuning disagreement: the pure-v2 receive handler confuses a requested block extent with its complete piece extent. Two newly executed 32 KiB-piece tests fail on the original implementation. Another new test shows that its nominal request window does not pipeline two blocks within one piece. Existing v2 fixtures largely exercise 16 KiB pieces, which conceal this geometry defect.

The right rebuild unit is the transfer and storage ownership model. More Tokio tasks, a different allocator, additional semaphores, or turning on io_uring cannot by themselves repair a torrent actor that waits for storage, a stop-and-wait transport, incorrect request accounting, or a flush operation that walks unrelated files. These techniques can help after the ownership and scheduling contracts are correct.

Confidence is high in the reproduced failures, allocation measurements, and source-level control-flow findings below. **Confidence is moderate** in the proposed architecture's policy choices. **Comparative speed gains are unknown** until controlled whole-client trials run. There is no defensible promise that any one implementation will dominate every workload: HDD locality, NVMe concurrency, low RAM, latency, energy, durability, and portability impose different tradeoffs. The product objective should be a demonstrably better efficiency frontier across declared hardware profiles.

2. What was verified

The audit traced the live v1/v2 receive and upload paths, request scheduling, uTP, piece selection, buffer ownership, file access, io_uring, recheck, secure move/copy, persistence, registry snapshots, scale tests, and benchmark scripts. Three parallel code reviews were reconciled with direct source inspection and focused execution. This is a performance and correctness audit of critical paths, not a formal proof of every protocol or security surface.

Executed evidence

Check	Result	Interpretation
Original `rt-piece-map`, `rt-piece-picker`, and `rt-session` tests	118 passed	Existing focused behavior remains a useful baseline; passing tests do not cover the new cases below.
New v2 32 KiB piece, memory assembly	Failed: piece read length does not match map	Full assembled piece was compared with the last block's extent.
New v2 32 KiB piece, disk fallback	Failed: Merkle hash mismatch	Completion read/hash used the last block rather than the complete piece.
New v2 within-piece request-window test	Failed: one outstanding request instead of two	A pending first block prevents filling a second block of the same piece.
Narrow extent correction, initial focused v2 suite	27 passed	25 existing tests plus the two new completion regressions pass.
Expanded extent correction suite	30 passed	Adds three test functions covering nonzero piece/file offset, short final block, and reversed arrival, each in memory and disk mode. The independent pipeline regression is explicitly excluded and remains unresolved.
Release allocator probes against original public APIs	Results below	Requested allocation bytes and events, not RSS or physical memory.

The temporary toolchain was Rust 1.99.1, with the repository lockfile retained. Builds used bounded jobs and reduced debug information for the engine test build. No controlled cross-client saturation test, real HDD/NVMe trial, or energy test was run in this environment. Its `/proc` process identity/metrics were not reliable enough for fresh RSS/CPU claims. Allocation probes use a Rust counting allocator and remain independent of that limitation.

Measured allocation requests

Operation	Estimate, where applicable	Allocation requests observed
Construct picker for 100,000 pieces	503,048 bytes	800,000 bytes, five allocations
Construct picker for 1,000,000 pieces	4,440,536 bytes	8,000,000 bytes, five allocations
Add one full peer to the 1,000,000-piece picker	Not included in the preceding constructor measurement	11,618,752 additional bytes, 166,665 allocations
Reuse unchanged 100,000-torrent snapshot	Cached	Zero allocations
Change one row and rebuild that snapshot	No physical-allocation claim	40,746,421 gross allocation bytes, 711,038 allocations; 39,500,040 net retained bytes across the measured operation

The picker constructor plus first full peer therefore requested 19,618,752 bytes in the measured structures, compared with a 4,440,536-byte estimate function. Peer input bitfield construction was outside the measured phase. Do not interpret this as a measurement of the complete process or a claim

that no other resource leases exist. Snapshot wall time was about 83.6 ms in one run; it is diagnostic, not a portable latency benchmark. Raw outputs, source, commands, and test logs accompany this report.

3. Transfer and protocol findings

F01 - Pure-v2 multi-block completion is broken

Priority P0. Confidence high; reproduced. `handle_v2_piece` obtains `region` from `validate_request(piece, begin, expected)`. That API correctly describes only the requested block, including its offset and length. The handler then uses it for complete-piece assembly, disk fallback reads, length checking, hashing, full-piece persistence, and progress. In the memory case a 32 KiB assembly is compared with a 16 KiB block length. In the fallback case only the final block is read and hashed. Simply deleting the length guard would be dangerous: the full piece could then be written starting at the last block's offset. v2 handler; v2 map contract.

The included proof fix separates `block_region` from the whole `piece_to_file` result, preserving block coordinates for spill writes and piece coordinates for completion. It fixes the two reproduced cases. Expanded tests also pass for nonzero piece offsets, a short final block, and reversed arrival in memory and disk modes, including full-file sentinels and whole-piece completion accounting. This does not complete the transfer rewrite or qualify production interoperability. The permanent design still needs distinct block/piece types, broad large-file coverage, and independent peers. Existing fixture construction hardcodes 16,384-byte pieces, explaining why a 32,768-byte fixture can still miss this bug. v2 fixtures.

F02 - The v2 request window cannot fill within a piece

Priority P0. Confidence high; reproduced. Request filling asks assembly for its first not-yet-received block. If that block is already outstanding, it skips the piece instead of finding the next unreserved block. The nominal 32-request limit can therefore distribute one request across each of many pieces but cannot pipeline two blocks of a single piece. This widens the working set while leaving a one-piece transfer effectively serialized. v2 request filling.

Selection must distinguish received, reserved, on-wire, expired, rejected, and completed blocks. Reserve all selected requests atomically against shared content ownership before the writer accepts them. Fixing this local loop is an urgent repair; the shared ledger in the transfer design is the durable solution.

F03 - V2 peers do not share authoritative acquisition state

Priority P0. Confidence high from source; duplicate-byte magnitude unmeasured. Outstanding requests and assemblies are created inside each v2 peer session. Peers can select the same missing pieces without a torrent-wide reservation owner. Disconnect, choke, request expiry, verification, and endgame do not share the lifecycle discipline available in parts of the v1 path. The peer audit provides the full call trace. v2 peer loop and state.

Use one ledger per torrent, with wire obligations indexed by connection and canonical content ownership shared across adapters. A socket address is not a connection identity. Reconnection and recheck require generations so late events cannot mutate a replacement. Internal generations are not transmitted on the BitTorrent wire: a late response with the same coordinates cannot magically identify which retry it belongs to. Keep unresolved obligations/tombstones or terminate them before tuple reuse.

F04 - V1 receive processing parks its torrent actor behind disk, hash, and persistence

Priority P0. Confidence high; live control flow traced. The main torrent select loop awaits its event handler. The piece handler can await a block write, download accounting, progress persistence, complete-piece verification, and completed-piece writes before it returns and refills requests. A worker running elsewhere does not make this torrent actor available while it awaits that worker. This is per-torrent serialization; it does not mean the whole daemon has one blocked CPU thread. actor dispatch; receive handler.

The new core must emit bounded jobs and process completions as later events. Independent pieces, disk operations, and peers can then overlap without spawning an unbounded task per block. Disk slowness should consume a declared work/byte budget and reduce admission, while control, timeouts, choking, shutdown, and already-ready completions remain responsive. Existing detached upload reads are a useful partial precedent; extend the principle consistently.

F05 - V1 uses a fixed 512 KiB peer flight window

Priority P1. Confidence high for the limit; workload effect conditional. Thirty-two 16 KiB requests allow 512 KiB in flight per peer. Memory pressure reduces this to eight requests, or 128 KiB. There is no effective BDP adaptation tying useful delivery rate and response delay to that window. request-window constants and pressure policy.

As arithmetic, 512 KiB divided by 50 ms is 10 MiB/s per peer before processing overhead. This is an illustrative window limit, not measured TorrentNG throughput; many peers and different RTTs change the outcome. Use an adaptive window clamped by remote `reqq`, useful work, actual memory commitments, and downstream service capacity. Rising disk backlog must shrink admission, not inflate the latency estimate and cause an even larger window.

F06 - Valid upload requests encounter a fixture-sized service ceiling

Priority P0. Confidence high. The peer loop accepts at most 4,096 requests in a ten-second window, then silently drops additional requests. With 16 KiB requests, that is 64 MiB per ten-second window: 6.4 MiB/s of sustained accepted payload per continuously used peer connection, subject to burst/window boundaries. The code comments tie the value to a 64 MiB fixture. guard constants; guard implementation.

This is a source-derived bound, not an observed aggregate upload measurement. Multiple peers or reconnects change total service. The repair is not unlimited request acceptance: use valid-range checks, bounded byte/job queues, fair resource admission, and protocol-correct rejection when negotiated. Ordinary high-rate demand should not be classified by a benchmark-derived message-count ceiling.

F07 - The uTP writer is stop-and-wait

Priority P0 for uTP qualification. Confidence high. The writer sends one chunk, waits until all sent packets are acknowledged, and only then advances to the next chunk. Although congestion/window state exists, it does not produce a useful sliding send window in this loop. At a 1,352-byte payload and 50 ms RTT, the arithmetic ceiling is about 26.4 KiB/s before losses and overhead. This is an illustrative bound, not a network benchmark. uTP send loop.

The receive path also discards out-of-order packets rather than maintaining a bounded reorder buffer. The send path's default I/O timeout can exceed the outer peer write timeout. Audit the entire transport state machine, not just the chunk loop: independent receive/ACK progress, multiple in-flight packets, congestion control, retransmission scheduling, SACK handling, sequence wrap, FIN/reset, connection demultiplexing, and bounded reorder memory must work together. Evaluate reuse of a maintained

interoperable implementation before committing to another custom one. uTP compatibility cannot be replaced with an unrelated transport. uTP state; BEP 29.

F08 - Fast-extension negotiation and request termination do not agree

Priority P0. Confidence high from the protocol and live handlers. V1 advertises Fast while its queue-full/choked upload paths drop requests, choke transitions clear pending upload work, and remote choke drains download requests. There is no outgoing Cancel variant in the v1 peer command enum. The v2 handler ignores incoming Cancel. These are not merely missed bandwidth optimizations; negotiated request obligations require a complete state machine. Fast advertisement; peer commands; upload loop; v2 peer handling.

BEP 6 requires a negotiated request to terminate with Piece or Reject and changes the meaning of choke/cancel. Apply the complete negotiated branch; distinguish legacy behavior and v2's own messages. Endgame must cancel redundant owners when one result wins, retaining enough state to handle late replies safely. Advertising only a convenient subset of a capability bundle creates interoperability debt. BEP 6.

F09 - Per-torrent upload rates are copied into per-peer buckets

Priority P0. Confidence high. Each peer receives the torrent's upload limit and initializes its own bucket from it. Unless the global limiter intervenes, several peers can each spend a full torrent allowance. The setting's aggregate semantics and its implementation differ. peer upload context; per-peer bucket.

Implement a hierarchical grant: global, then torrent, then peer fairness. A grant succeeds atomically across required levels. Updating a rate must not mint a fresh burst in every connection. Measure both raw network cost and useful payload; retransmission and duplicates remain real costs even when not counted as useful progress.

F10 - Webseed download accounting is hidden inside `debug_assert!`

Priority P0. Confidence high from source. The local token debit appears as `debug_assert!(self.try_consume_download_tokens(req.length));`. That expression does not run in ordinary release builds. Debug testing can therefore validate behavior that the release binary does not execute. webseed request admission.

Perform the mutation in normal code, handle admission failure explicitly, and assert only an already-computed result. Add a release-mode rate regression. A grep rule against side effects inside debug-only assertions can supplement that behavioral test, but does not replace it.

F11 - Download pacing lacks a reliable token-ready wakeup

Priority P1. Confidence high for the missing scheduling mechanism; stall conditions depend on other events. Request refilling is driven by peer/other activity. If a low rate exhausts available tokens and outstanding responses drain, the client can lack the event that would restart useful requests when tokens become available. Other activity may incidentally unstick it. This makes a rate limit partly an event-liveness problem. peer request refill.

Schedule the earliest eligibility deadline with an injected monotonic clock. Rates below one block per second must still progress. Do not solve this with a millisecond polling task per peer; one fair scheduler should wake only eligible work.

F12 - Choking policy uses the wrong feedback and weak exploration

Priority P1. Confidence high for implementation; swarm effect unmeasured. The choker ranks peers using uploaded-byte rate even for download-oriented reciprocity, and optimistic selection can repeatedly choose the first eligible idle peer. The last sampled rate can remain stale while a peer is idle; the choker does not apply the display path's idle decay. choker; rate sampling and display decay.

Separate downloading and seeding policies. Use useful received contribution while downloading, actual service and fairness while seeding, rotating/randomized bounded exploration, and time-decayed observations. Keep the policy pure enough to replay deterministic swarm traces. V2 should share the same deliberate slot policy instead of unchoking every interested peer.

4. Memory, CPU, and storage findings

F13 - Memory pressure can throw away the only copy of received piece data

Priority P0. Confidence high. V1's aggregation path reserves a full piece-sized assembly when the first block arrives; oversized pieces and restored partial pieces have a separate path. Budget enforcement can evict another unfinished assembly, reject its picker state, and request that piece again. Comments acknowledge the discarded copy. The mechanism bounds a particular memory container by spending additional network bytes, bookkeeping, and time. assembly admission and eviction.

Reserve potential receive capacity before requesting data, concentrate work on active pieces, and spill already-received data through a bounded path when necessary. Memory commitments, resident capacity, and speculative cache are different quantities. A torrent whose device cannot drain should stop acquiring new bytes without causing all other devices to stop.

F14 - Picker memory accounting assumes packed `Vec<bool>` storage

Priority P0 for trustworthy limits. Confidence high; measured. The estimate budgets three packed bitsets, but the actual fields are `Vec<bool>`, whose elements occupy one byte in this Rust implementation. Availability also owns a retired vector and ordered buckets. At one million pieces, construction requested 8 MB against the estimator's 4.44 MB. Adding the first full peer requested another 11.62 MB. picker fields, estimate, and constructor; availability representation.

Use actual packed bitsets where appropriate and account capacities of every owner, including bucket/index allocation. Avoid an O(number of peers x pieces) bitmap expansion beyond the declared budget. A memory governor is only as good as the quantities it reserves; narrow payload counters must be named as narrow counters.

F15 - The common picker path scans all pieces before using rarity buckets

Priority P1. Confidence high for complexity. `pick` first searches all piece indexes for a positive-priority candidate. In the common all-default-priority case it scans the full piece space on every selection before reaching its rarity structure. `pick_from_seed` has the same pattern. Peer bitfield changes also relocate entries through `BTreeSet` rarity buckets, producing the measured allocation storm. picker selection; availability updates.

Maintain explicit nonempty priority/rarity indexes, with compact counts and stable array/slab ownership. Choose a structure by measured update/select workloads; an array-backed bucket design is a candidate, not a proven current feature. Randomize equal-rarity ties without scanning or allocating on each block. Benchmark one million pieces, full seed joins, churn, partial availability, streaming priority, and endgame separately.

F16 - Buffers cross boundaries without retaining one coherent owner

Priority P0 for the new contract, P1 for individual copies. Confidence high. Ordinary frame-to-Bytes conversion can transfer the Vec without a payload copy, but drops pool return/accounting ownership. Registered conversion copies. The fixed-buffer read path can allocate an ordinary frame before acquiring a different registered slot. The default 512 KiB read size also misses the pool's 16/64/256 KiB classes. Elsewhere completed-piece verification copies assembly, v1 writes copy mapped ranges, and upload coalesces even a single read into a new Vec. frames; fixed read allocation; v1 piece copies; upload read copies.

Use an immutable owner retained through every `Bytes` clone/slice and through backend completion. The pinned `bytes` 1.11.1 already supports `Bytes::from_owner`; an upgrade is not needed for that mechanism. Charge allocation capacity once until the final consumer and kernel operation release it. Distinguish physical capacity from work-in-flight limits. A deliberate cross-file coalescing copy can be reasonable; API-boundary copies without purpose are not. Bytes ownership API.

F17 - The shared file cache serializes filesystem metadata work

Priority P1. Confidence high. A hit takes the shared cache mutex, sweeps expired entries across the cache, and verifies pathname identity. A miss opens the file while holding that mutex. Exact access-mode matching can discard a read/write descriptor for a read-only request and reopen it later for writing. Upload range-cache validation adds another file-stat operation. Across many files or a slow network filesystem, a supposedly warm hit still does metadata work under common contention. file cache; identity check; read-cache validation.

Create one secure file catalog, reuse compatible descriptors, use incremental/periodic expiry, and open outside shard locks with race-safe publication. Preserve no-follow, path confinement, replacement handling, and namespace generations. Faster managed-payload access requires an explicit ownership contract; filesystem watchers alone do not prove unchanged content. Preserve conservative per-access checks when externally mutable payloads require that behavior, and report its cost separately.

F18 - Default readahead can turn random 16 KiB demand into 512 KiB reads

Priority P1. Confidence high for logical backend amplification; physical disk effect unmeasured. A miss may expand a block to 512 KiB, while the cache retains only one window per file. Random or competing readers can replace useful windows before reuse. Memory for the cache may be denied only after the enlarged read has occurred. The live peer-read elevator is bypassed when readahead exceeds the requested length, which is true for the default block size. read admission and expansion; range-window storage.

With no reuse, 512 KiB/16 KiB is 32 times the demanded bytes at the backend interface. Page cache and filesystem behavior decide physical I/O; do not label this a measured 32x disk slowdown. Pre-reserve speculative capacity, merge actual nearby pending requests first, deduplicate in-flight ranges, and grow prefetch only after observing useful reuse. Exact reads are the low-memory fallback.

F19 - A torrent flush can sync unrelated open writable files

Priority P0. Confidence high. `sync_all_open_files` starts with the scheduler's dirty generations, then unions the process-shared file pool's writable paths and syncs them serially. A torrent checkpoint can therefore touch clean or unrelated files, potentially across devices. Per-torrent scheduler ownership combined with shared descriptor ownership amplifies barriers rather than defining precise coverage. flush enumeration and sync; shared resources.

Track per-file write sequences and per-torrent snapshot watermarks. A barrier waits for exactly the writes it covers, syncs affected dirty files, returns coverage evidence, and leaves newer writes dirty.

Coalesce compatible barriers. Retain the existing generation check's purpose and the existing durability modes; removing fsync to win a benchmark is not a repair.

F20 - io_uring is operated as a batch barrier

Priority P1. Confidence high. Each io_uring worker receives a batch, submits it, waits for all submitted operations, then processes completions and accepts more work. An unrelated slow operation in that worker's batch can postpone publishing already-completed reads and refilling available queue slots; other workers may continue. File registration can also churn at idle boundaries. Positive short I/O is treated differently from the portable exact-I/O loop. The latter is a safe failure discrepancy, not established silent corruption. ring driver; registration and completion.

Continuously harvest CQEs, complete each operation independently, and refill useful work as slots free. Handle short reads/writes by advancing the suffix when required by the operation contract. Retain file registrations with generations and lifetime limits. Ordered durability still requires covered write completions before fsync; asynchronous submission does not create that ordering automatically. io_uring manual.

F21 - Cancellation may release admission and dirty evidence before issued I/O resolves

Priority P0 investigation. Confidence moderate: ownership risk, not reproduced data loss. Some permits belong to the caller's async write future, while the backend can continue to own submitted data. Dropping the waiter may return its admission credits early. Dirty marking follows an awaited completion, while the detached watcher mainly invalidates cache state. The exact cancellation interleavings need a deterministic fault test before a stronger claim. scheduled write lifetime; completion watcher.

Accepted operations must own permits, file references, payload, attempted ranges, and completion records until the service confirms their terminal state. Cancellation before issue differs from loss of interest after issue. Reserve completion capacity at admission so a full actor mailbox cannot discard write evidence or deadlock the worker that would release memory.

F22 - Storage has several scheduling systems without one live fairness authority

Priority P1. Confidence high. The generic weighted `DeviceElevator` is exercised by its tests but not the main live request dispatch. Real scheduling combines class semaphores and shared device bounds, with a different peer-read coalescer. Move/copy execution also escapes the ordinary class admission path. Concurrency caps do not by themselves provide cross-torrent fairness or prevent recheck/copy from competing outside the same policy. generic elevator; scheduler; move execution.

Use one actual service per contention domain for peer reads/writes, recheck, proof construction, preallocation, copy, and sync. Device grouping must represent shared physical/server resources, not merely mount names. HDD locality and NVMe concurrency require different policies behind the same deadlines, fairness, and ownership contracts. Cross-device copies reserve both sides in a deadlock-safe order.

F23 - V2 recheck repeats small I/O and task overhead for every Merkle leaf

Priority P1. Confidence high. Recheck reads a 16 KiB leaf, performs sparse-range work and possible dense copying, then submits hash work before continuing to the next leaf. A dense GiB contains 65,536 such leaves. The v1 verifier already has larger chunk/double-buffer work worth preserving; do not rewrite it into the same mistake. v2 verifier.

Read bounded multi-leaf chunks, maintain sparse cursors, hash several leaves per worker turn, and return batched results. Hash CPU and disk service receive independent bounded admission. Hybrid content should share a read stream while applying both correct hash views; v1 and v2 padding/boundary semantics cannot be assumed identical.

F24 - V2 proof serving rereads the whole file and imposes a 64 MiB ceiling

Priority P0 for complete v2 support, P1 for performance. Confidence high. A hash request can reserve/read the whole file and rebuild its Merkle tree; files above the hard-coded 64 MiB safety bound are rejected by that path. A safety cap has become a functional limitation because the work unit is wrong. proof construction.

Maintain a bounded, versioned store of authenticated nodes/piece layers keyed by content identity and tree coordinates. A hot proof should read no payload bytes; a missing proof should trigger bounded subtree work. Never treat hashes computed from unverified on-disk content as authenticated expected hashes. Raising the file-size cap is not an acceptable replacement for this design. BEP 52.

F25 - Secure copy re-verifies leaves at each ancestor

Priority P1. Confidence high for recursive work. The Unix secure-copy routine verifies a copied entry; copying a directory recursively calls that same routine for children and then recursively verifies the directory again. A leaf beneath d copied directories can be hashed on source and destination d+1 times, in addition to copy I/O. Deeper trees multiply work. This finding is scoped to the inspected Unix implementation. The security objective is valid; the repeated traversal is avoidable. secure copy recursion; recursive verification.

Return verified child manifests/evidence and aggregate them at the parent, with generation/descriptor bindings that preserve substitution checks. Schedule copy reads/writes in the same device domains as transfers. Retain no-overwrite behavior, confinement, rollback journals, and namespace durability; these are some of the repository's strongest foundations.

5. Control plane, tests, and evidence findings

F26 - One changed registry row rebuilds a deeply cloned global vector

Priority P1. Confidence high; allocation probe executed. Per-shard caching exists, but the combined snapshot extends one vector with cloned entries from every shard and sorts it. Cloning entries clones owned strings. Changing one row in a 100,000-row registry triggered approximately 711,000 allocations and 40.7 MB of allocation requests. An unchanged snapshot allocates nothing, so the defect is invalidation/rebuild cost, not every cache hit. registry snapshot.

Native and qBittorrent APIs have genuine incremental `ChunkedVec` paths; preserve them. Full rebuild/resynchronization and several compatibility paths still flatten the registry. Publish stable immutable chunks and revision deltas from the registry itself, keep cold metadata separate from changing counters, and let paging/compatibility adapters materialize only what they actually need. native incremental path; compatibility consumer.

F27 - Dormant-memory estimates omit most ownership, and timer cancellation has a quadratic case

Priority P1. Confidence high for code/complexity; production frequency of worst case unknown. `DormantTorrentSnapshot::estimate_heap_bytes` returns the infohash string capacity. It excludes map buckets, inline values, duplicate identity keys, timer structures, and other runtime owners. Several scale examples use literal arithmetic budgets. Separately, the named timer wheel stores shared-deadline deques; cancel uses `retain` across a deque. Repeatedly canceling N entries with the same deadline is quadratic. For 100,000 entries the arithmetic is about five billion key comparisons. dormant estimate; timer implementation.

Use compact generational identities throughout runtime maps, accurately name narrow projection counters, and reconcile whole-owner accounting with allocator and OS measurements. Use indexed unlink or a generational heap with bounded stale-entry cleanup. Dormant torrents should have bounded compact state and shared scheduling, not hidden per-torrent loops. Preserve bounded due-pop behavior that already exists.

F28 - Every engine command can scan all active task handles

Priority P1. Confidence high for complexity. Before handling each engine command, the actor reaps finished torrent tasks. The helper iterates the task map, filters `is_finished`, and only then applies a cap on the number of matches. With no finished tasks it still examines the entire active map; the cap bounds results, not search work. The health tick repeats the same scan. command dispatch; reap helper; reap use.

Use a supervised completion queue or a join set whose completions identify terminated actors. Retain bounded cleanup per turn and failure-isolation behavior. A periodic low-frequency reconciliation may remain a fallback; it should not impose O(active torrents) work on every ordinary command.

F29 - A metrics scrape can construct an additional storage runtime

Priority P0 for trustworthy telemetry. Confidence high. The metrics handler calls `StorageRuntime::global`. Its lazy initialization creates a handle cache, selected backend, and sweeper thread, while live torrents use separately shared MountScheduler resources. A scrape can therefore add resources and report a backend that is not the live transfer path. metrics handler; runtime initialization.

Inject read-only references to existing live service counters. Observability must not construct the subsystem it claims to observe. Inventory metrics by owner, explicitly separate capability from active use, and expose bounded queue age, admitted bytes, wasted traffic, cache usefulness, dirty coverage, and outstanding kernel operations.

F30 - Engine unit tests replace the production database boundary

Priority P1. Confidence high. Under `cfg(test)`, database operations use a direct mutex path instead of the production worker; batched operations also have different execution/transaction behavior. Passing those tests cannot establish production worker queueing, batching, ordering, or cancellation behavior. This does not mean no integration/crash tests exist; they do. database worker variants; engine DB boundary.

Use dependency injection to control time and faults while exercising the production ownership path. Keep small synchronous database unit tests, but make worker lifecycle, batching, slow transactions, failure completion, crash boundaries, and engine responsiveness run through compiled production services.

F31 - Scale checks certify synthetic projections, sometimes after dropping the application

Priority P0 for release claims. Confidence high. The 100k router test passes its only router by value to a function that consumes the request and returns a duration, then samples resources. A retained allocator footprint is not live application ownership. The fixture contains synthetic API rows, not a production engine with persisted metadata, scheduling, storage, and recovery. Missing metrics can become zero or skipped assertions. Another 1k hot-memory check fabricates stats and sums only the ten displayed rows. router scale test; top-ten test.

Keep these as API/reporting smoke tests and label them accurately. Current documentation already calls some checks proxies; the audit is not accusing it of hiding that fact. Add real live-handle barriers, production import/restore, complete resource counters, and invalid results when required measurements are unavailable. A top-ten attribution panel is never an aggregate memory bound.

F32 - Existing throughput comparisons do not establish capacity or fair completion time

Priority P0 for optimization decisions. Confidence high. The main script defaults to 32 MiB/s for clients and 4 MiB/s for the controlled receiver; both are environment-overridable. The receiver default is below the upload request guard. Download timing starts after add and excludes external validation, while seed timing starts after the receiver is running and includes resource sampling plus external checksum at completion. Synchronous sampling, including `docker stats` and other probes, makes a nominal two-second cadence roughly 3.6 seconds in checked-in raw data. benchmark caps; seed timing; download timing; sampling.

Preserve a capped matched-goodput experiment, but add unthrottled capacity trials with proven helper headroom. Use one release barrier, exact declared completion events, separate verification/flush clocks, monotonic cumulative counters, randomized paired repetitions, and invalidation rules. No later binary inherits qualification from a historical report.

F33 - CPU, cache, and backend microbenchmarks can reward the wrong behavior

Priority P1. Confidence high for methodology; effect sizes mostly unknown. The process sampler replaces aggregate CPU with the sum of surviving threads, losing exited workers' work. Mean sampled CPU percent confuses idleness with cost. Seed preparation hashes payload and prefers hard-link reuse, with copy fallback, so it does not establish cold independent-I/O behavior. A storage microbenchmark exposes an entire shuffled-adjacent file to the elevator at once. Both arms disable readahead and allow all-block concurrency; the treatment enables 25 ms coalescing while the baseline disables the elevator. This demonstrates an ablation under unusually favorable arrival exposure, not independent comparative capacity. Task/validation work is also inside timing, and separate queue-depth-one backend loops do not measure asynchronous capacity. CPU sampler; seed preparation; storage microbenchmark.

Use CPU seconds and joules per verified GiB, complete charged memory plus its categories, physical and logical I/O amplification, latency, and verified completion. Separate warm/cold data and independent inodes. Sample cumulative cgroup counters with an external observer; networking work outside that cgroup needs whole-host accounting when comparing total cost. Linux explicitly documents shared page-cache charging and cumulative CPU/I/O counters, so a single anonymous-RSS or Docker percentage cannot represent all resource consumption. Linux cgroup v2.

F34 - Certification can associate current source with historical PASS text

Priority P0 for evidence provenance. Confidence high. The memory-roadmap certification script selects reports by filesystem modification time, searches text for successful test names, and stamps the generated summary with the current commit. It does not prove that the selected results belong to that source, binary, configuration, or workload. Some missing storage evidence becomes WARN without failing the aggregate result. The script explicitly limits torrent-count claims, but that disclaimer does not make its evidence identities match. certification aggregator.

Generate qualification from hash-verified machine manifests keyed to exact source tree, lockfile, binary, workload, environment, and parser schema. Missing or mismatched required evidence means not qualified. Markdown is a view of validated results, not an input parser for release truth. Historical evidence remains useful under its original identity.

F35 - A successful announce waits for an optional scrape before delivering peers

Priority P1. Confidence high for control flow; added latency conditional on the scrape. After a successful HTTP tracker announce, TrackerWorkers awaits a scrape before sending the announce result and peer list back to the engine. It does so even when announce counts are already present. The scrape performs another validated resolve and HTTP request; with the default 30-second HTTP timeout, optional tracker statistics can hold up usable peers. tracker announce completion; scrape request; HTTP timeout.

Publish the successful announce immediately. Coalesce scrape work behind a separate low-priority cap and return generation-checked statistics later. Test fast announce with a stalled scrape. This is not a claim that tracker connections lack pooling: the existing egress code caches clients after DNS revalidation. DHT also has aggregate rate/query/peer caps worth preserving.

6. What the checked-in performance results actually establish

The October 13 report records TorrentNG at 14.12 MiB/s and qBittorrent at 17.18 MiB/s in one ext4/public-swarm download; four started clients passed external hashes. The October 14 NFS report is much worse: TorrentNG/system at 0.49 MiB/s and qBittorrent at 16.82 MiB/s, with a separate TorrentNG allocator run not completing within its window. These are serious historical warning signals. October 13 report; October 14 report.

They are not current-HEAD rankings. The October 14 system result identifies a different dirty source revision and fingerprint; allocator, swarm, storage, and source conditions are confounded. Do not advertise a stable 34x deficit or claim that switching allocators caused a proven improvement. The reports themselves acknowledge limited confidence. The rTorrent image did not start; failed setup is not a zero-throughput competitor. The new suite must repair that comparator and add rqbit, while recording the embedded libtorrent version for qBittorrent/Deluge.

The engineering conclusion is stronger than any disputed benchmark ranking: multiple current-code mechanisms necessarily waste or restrict useful work, and the present tests cannot determine whether changing them improves the whole product. That is enough to justify the rebuild below without inventing a speedup.

7. What to retain, replace, and defer

Retain and test: secure descriptor-relative path handling, validated wire/metainfo parsing, v1/v2 mapping rules, no-overwrite move/rollback journals, explicit crash/durability policy, useful bounded channels, the existing portable exact-I/O loops, dedicated production DB worker, v1 chunked verification work, API adapters, and incremental API snapshot machinery. Retention does not exempt code from a bug fix; it prevents destroying valuable behavior in a rewrite.

Replace at the architectural boundary: per-peer v2 acquisition state; actor-awaited payload work; guessed memory accounting; disconnected device schedulers; pool owners that lose lifetime accounting; full-registry flattening; scan-based task completion; and benchmark clocks/qualification rules.

Defer until measured: global allocator selection, unsafe custom buffer initialization, mmap/direct-I/O as universal defaults, SQPOLL, sendfile across every transport, and target-specific SIMD tuning. Rust enables safe ownership and efficient representation. It does not make redundant copies, O(N) scans, or serialized protocols efficient automatically. io_uring and zero-copy are implementation tools with lifetime/order constraints, not product outcomes.

The companion `ARCHITECTURE.md` defines the new boundaries and rollout. `BACKLOG.json` is the dependency-ordered implementation contract. The storage, transfer, and benchmark specifications supply detailed semantics and acceptance tests; the regression bundle makes the first failures independently reproducible.

Part II - Replacement architecture

Status: an implementable rebuild specification, not a claim that the replacement engine already exists. It targets audited revision `05d5c23886446f380552c73e0be480f03ae2d2ab`. The included local code patch fixes one reproduced v2 extent defect. The remaining work is explicitly tracked in `BACKLOG.json`.

1. Product contract

The objective is maximum verified useful transfer under declared resource and service constraints. A client is not more economical merely because it is idle more often, reports a smaller partial memory counter, or reaches a higher short-lived network peak. It must finish the requested content correctly, preserve its stated recovery behavior, and spend less resource per useful byte at comparable service levels.

Measure six outcomes together: verified and durable completion time; CPU seconds per verified GiB; complete charged memory and its anonymous/cache/kernel components; physical and logical I/O amplification; energy per verified GiB where instrumentation exists; and responsiveness/fairness while the engine is busy. Report startup/metadata time and small-torrent tail latency separately from long sustained transfers. A single synthetic score must not conceal a disastrous workload.

Use product-default and equal-resource comparison tracks. Use both capacity and matched-goodput trials. At a matched rate, a client that misses the setpoint fails that operating point; it does not win a low-CPU award. At saturation, identify the limiting resource before interpreting a speed result. Equal durability is required for a durability comparison; ordinary product-default completion remains a separate event.

The first release goal is correctness and elimination of the reproduced structural waste. The competitive goal is a measured frontier across HDD, SATA SSD, NVMe, NAS, low RAM, and large-catalog workloads. Numeric proposed gates live in the benchmark contract and are not achieved results.

2. Ownership boundaries

Keep the current crate boundaries unless a dependency cycle requires a small extraction. A new directory is useful when it establishes a real ownership contract; creating dozens of crates does not establish one by itself.

Owner	Responsibilities	It must not do
Session supervisor in `rt-engine`	Lifecycle, configuration, admission across torrents, service ownership, restart and shutdown	Copy payload buffers, synchronously scan all task handles per command, or execute long jobs inline
Pure transfer core in `rt-engine/src/transfer/`	Geometry adapters, authoritative request/content ledger, picker policy, protocol obligations, piece evidence	Own sockets, file descriptors, SQLite, sleeping futures, or unbounded tasks
Peer runtime / `rt-peer-wire` / transport	Bounded parsing, independent receive and ordered send progress, wire completion events	Decide global content ownership or silently abandon negotiated requests
Device service in `rt-storage`	Secure file catalog, all actual I/O admission, read/cache policy, write sequences, barriers, completion retention	Lose ownership because a caller canceled or enumerate unrelated files to define durability

Owner	Responsibilities	It must not do
CPU service in `rt-hash` plus runtime adapter	Bounded hashing jobs over immutable segments, authenticated Merkle work	Create one scheduling unit per tiny leaf or duplicate payload solely to cross an async boundary
Persistence worker in `rt-db` / engine boundary	Ordered batched transactions, coalesced projections, checkpoint evidence	Invent clean payload state before covered writes are durable
Read model in `rt-session` / `rt-api-model`	Stable chunks, revision deltas, compact metadata identities, bounded snapshot versions	Deep-clone all cold strings because one rate counter changed
Discovery/control services	Trackers, DHT, DNS, metadata exchange, proxy/auth control, logging and observability	Consume payload progress reserves or trigger unbounded wake/retry storms

The core consumes timestamped events and returns bounded actions. Actions refer to typed identities and owned data, not ambient paths or borrowed mutable assembly memory. Adapters execute actions and return completions. One core owner per active torrent is the initial model. Do not shard a single torrent until profiling demonstrates that the core itself is CPU-bound after full scans and copies are removed.

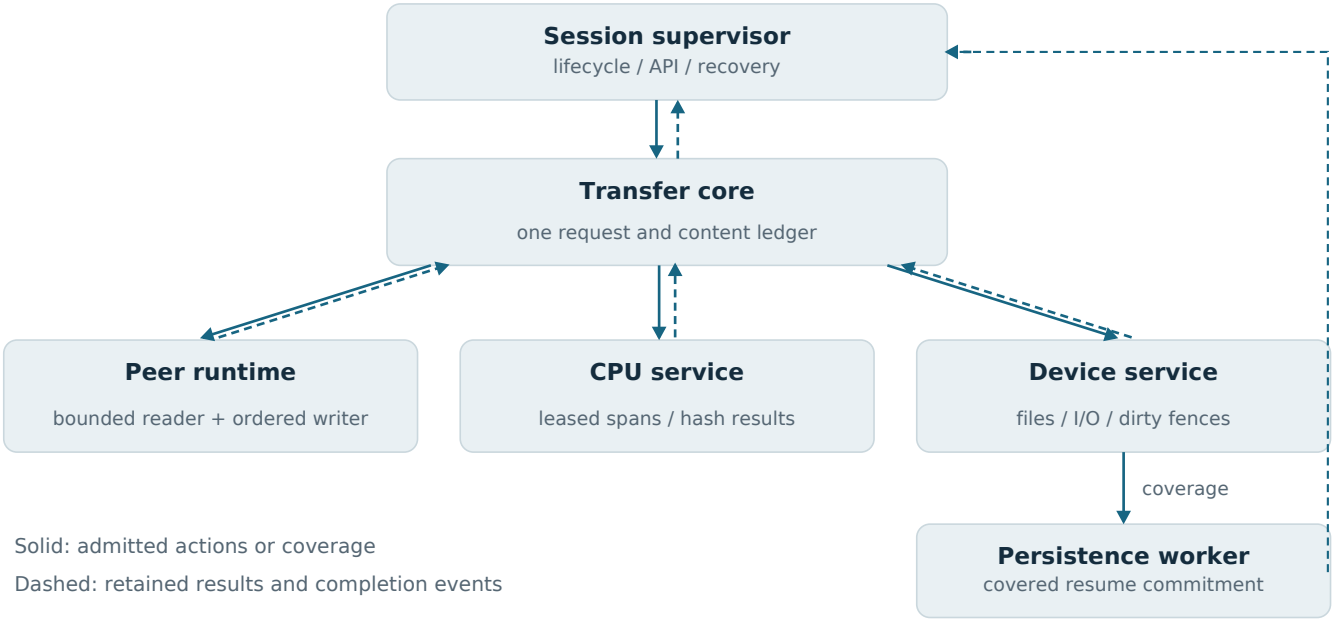


Figure 1. A core that emits bounded actions and receives retained completions. Payload ownership and durability evidence survive cancellation independently of a waiting actor.

3. The data path

Download

The ledger selects useful missing content and reserves metadata, expected receive capacity, and a bounded downstream work commitment. A peer writer accepts the request and reports its on-wire transition. The reader validates the header and matches coordinates before admitting the payload body. Received immutable spans enter the same content owner regardless of peer. Independent hash and device services work on admitted ranges, then send completions. The core updates evidence, serves/advertises eligible pieces, and immediately schedules more useful work when credits permit.

This is a pipeline, not a chain of awaits inside a torrent command handler. It is also not unrestricted concurrency: every queue, retained result, and pending body has a finite owner and budget. A slow disk consumes its allowed backlog and reduces admission for torrents using that device. It does not force unrelated devices to stop or let all peers keep filling RAM.

Block and piece extents are different types. V1 has concatenated-file and virtual-padding views. V2 has file-aligned piece space and Merkle requirements. Hybrid acquisition initially uses one download plane while both verification views are represented accurately; mixed-plane acquisition is a later capability gate. Equal piece numbers do not prove equal canonical content ranges.

Upload

Validate the request and its negotiated obligation, then enqueue a small bounded read job. Reuse compatible cached or in-flight ranges where useful. The writer receives a header and immutable payload spans, retaining a partial vectored-write cursor. Only one writer owns socket ordering. Once a Piece frame begins, control messages cannot be inserted inside its body; prioritization happens at frame boundaries. Read progress must remain independent of a slow outgoing socket or disk request.

Cancellation removes unsent work where possible and resolves races through the obligation state. Valid excess demand receives the protocol's supported rejection behavior or bounded legacy handling. A queue limit is not a reason to advertise an extension and then ignore its terminal-response contract.

Evidence states

Received bytes may exist only in memory or may already be spilled. Verification binds the exact content view to expected hashes. Write completion establishes a readable disk source. Servability requires verification plus a stable accessible source. Payload durability requires a successful ordered barrier over relevant writes and namespace operations. Resume commitment requires a durable state transaction carrying that coverage.

These are partly orthogonal states, not one simplistic sequential enum: direct-write mode can write before a whole piece is verified. The first implementation sends HAVE after verified write completion. Early serving from memory is a separate optimization requiring a pinned owner accessible to upload. A canceled task never supplies missing write or durability evidence.

4. Capacity follows the data

Use a retained immutable backing owner, for example `Bytes::from_owner(PayloadOwner)`, supported by the pinned bytes release. The owner holds the allocation, actual capacity charge, pool-return behavior, and any registration lifetime. Clones and slices share that owner until final release. Idle pool capacity still consumes memory. A tiny slice retaining a large slab is observable and may justify a deliberate compaction copy.

Track physical allocations, commitments, and service limits separately. An illustrative engine budget is:

```
resident owned capacity + committed capacity not already backed by a retained slot + metadata reserves + bounded progress reserve <= engine memory budget
```

Do not double-count the same resident pool slot as both an allocation and a future commitment. Do not omit capacity merely because data moved from the decoder to hashing or from a frame into Bytes. Device in-flight bytes and remote request counts are additional operational constraints; they are not additional physical copies by definition.

Kernel socket buffers, page cache, filesystem metadata, allocator overhead, and unrelated process/runtime allocations are outside a narrow payload governor. Expose them separately and

validate the total with a process/cgroup limit in real-host trials. The operating-system hard limit is a final containment boundary, not a substitute for graceful admission.

Prevent resource deadlocks deliberately. Reserve one completion slot for each admitted operation. Retain terminal results in a bounded operation slab even if an actor notification queue is full. Keep a measured progress reserve for writeback, barriers, teardown, and control. Never wait for a descriptor while holding a catalog mutex, wait for a second device indefinitely while owning the first device's scarce slots, or consume every memory slot with reads that cannot complete without writeback capacity.

5. Scheduling policies

Network and rates

A starting flight target is sustainable peer delivery rate multiplied by measured response delay plus a bounded local-turnaround margin. Clamp by remote request capacity, useful content, connection/torrent/global credits, device service allocation, and fairness. Measure response delay from actual request transmission, not initial queue insertion. A growing disk backlog reduces sustainable admission; it is not permission to enlarge the window.

A single hierarchical rate scheduler grants global, torrent, and peer shares with byte-weighted fairness and fractional credit. It has explicit eligibility deadlines, including rates below one block per second. Limit updates change shared state without replenishing every peer's burst. Requests reserve potential bytes, while receive pacing and raw-payload accounting express the documented download contract. Already-sent bytes in the kernel cannot be retracted by an application token bucket.

The pure core must handle event reordering between persistent reader and writer tasks. A response can arrive before the writer's `RequestCommitted` notification reaches the core. Match against the existing queued obligation, defer/reconcile the late commit, and prevent double debit or resurrection. Internal generations do not distinguish late same-tuple replies on the wire; connection/terminal semantics do.

Device and CPU

All peer reads/writes, recheck, proof construction, preallocation, move/copy, and sync enter the same actual contention service. Device grouping can include multiple mounts sharing one disk or multiple network shares sharing a server. Discover topology where possible and make unknown grouping explicit. Bound jobs by bytes and service time, not only count.

HDD policy uses locality and bounded merge opportunities with deadlines. SSD/NVMe policy continuously refills useful depth and avoids arbitrary seek-delay windows. NAS policy isolates slow metadata/latency and respects server limits. Low-memory policy reduces speculative reads and concentrates pieces. Each policy is tuned against live workloads; there is no universal cache size, queue depth, or worker count.

Use a bounded CPU service with appropriately large hash jobs. V2 leaves can be calculated within a multi-leaf chunk. V1 updates remain ordered within a piece. Hybrid verification shares data reads where possible but still performs the hashes required by both views. Hashing, sparse traversal, and proof work yield at bounded chunk boundaries so recheck cannot monopolize transfer CPU.

6. Storage and durability details

Start with the existing portable positioned-I/O loops as the reference backend. This keeps correctness separable from Linux-specific queue mechanics and gives every optimized backend a behavioral oracle. Add continuous io_uring only when the engine uses the new owner/completion path. Harvest completions promptly, retain operation resources through the original CQE, refill free slots, and retry short-I/O suffixes according to the operation contract.

A secure file catalog owns root capabilities, IDs, namespace/content epochs, expected shape, descriptor state, device identity, and dirty evidence. Reuse read/write descriptors for compatible reads; publish opens outside shard locks; use bounded expiry maintenance. Catalog IDs avoid repeated hot-path string/hash/path work. Managed engine-controlled payloads may use faster ownership-based access. Externally mutable payloads require an explicit conservative policy; watchers are hints, and same-inode edits invalidate verification just as replacement can.

Writes receive file and torrent sequence numbers before issue. Record attempted ranges before a waiter can disappear. A torrent checkpoint watermark maps to exact file fences. Wait for covered operations, sync only affected dirty files, coalesce compatible fences, and preserve newer dirtiness. Then durably commit the matching resume snapshot. File creation, replacement, and moves retain required directory/journal ordering. Weak or unsupported mount semantics stay explicit.

Content changes, pause/remove, recheck, and moves quiesce old-epoch work. The system stops new admission, resolves issued operations, applies the secure namespace transition, then publishes the new epoch. A stale write must not land after a move is considered complete. Rollback uses known durable state and conservative recheck when evidence is missing.

Do not promise hardware persistence beyond what the OS/filesystem/storage contract supplies. Also do not weaken the current durability policy to make a speed comparison look good. Report ordinary verified completion and a separately defined durable boundary.

7. Control plane and ancillary I/O

Session and API model

Intern stable external infohashes/names/paths at the boundary and use compact generational IDs internally. Keep cold metadata immutable and share it; store frequently changing counters separately. The registry publishes revisioned chunks and deltas without flattening every row. Compatibility endpoints may materialize a full response when required, but should not force every caller through that cost. Pagination, sort-key caching, filtering, and snapshot retention have explicit memory/time budgets.

Replace per-command full task scans with supervised completion events. Replace same-deadline cancellation scans with indexed unlink or bounded generational structures. Batch add/remove/pause/restore work at bounded turns, so 100,000 dormant records do not delay one active torrent's stop request for seconds. A low-frequency integrity sweep can remain outside the command path.

Persistence and control traffic

Keep the production database worker and make its queue, batch size, transaction duration, and failure state observable. Coalesce progress updates by torrent/revision; preserve lifecycle/error events that cannot be dropped. Never await a long transaction inside the session core merely to update a cosmetic rate. Recovery-relevant ordering remains explicit. Unit tests should inject dependencies while running the production worker path.

Discovery, DNS, tracker announces, DHT, metadata transfer, authentication/proxy setup, API serialization, and logging need their own queue and concurrency budgets. They share global memory/CPU containment but cannot exhaust payload-completion reserves. Coalesce redundant announce/DNS work, jitter retries, bound metadata bytes before parsing, and support cancellation/shutdown without one timer task per dormant torrent. Audit these paths with profiles and protocol fixtures before claiming the whole product is optimal; this review did not formally verify every discovery protocol.

Log volume is I/O too. Default transfer logs should record bounded state changes, failures, and aggregate diagnostics rather than one formatted line per block. Include logging and compatibility APIs in production workload profiles. Metrics must observe the existing services, never instantiate another runtime. Heavy smaps/stack/syscall analysis belongs in separate diagnostic trials whose observer cost is measured.

8. Migration without a dangerous rewrite cliff

Keep a reference implementation selectable at startup and route existing APIs through adapters. Choose one engine implementation for a payload ownership domain. Never run old and new writers against the same files. Read-only shadow comparisons may inspect policy decisions; they must not generate duplicate network requests. Fake backends and independent fixture roots are the places to compare write traces.

Preserve database and resume compatibility initially. New hash caches are versioned/disposable. A state-format change needs its own migration and rollback tests, including crash between every publication boundary. A feature flag is not a rollback plan unless old code can actually consume the durable state left behind. Export a known durable checkpoint before switching, and conservatively recheck uncertain ranges on rollback.

Converge v1 first through the new core and portable service, then pure v2 and hybrid adapters. Shared contracts land before two teams build incompatible ownership assumptions. uTP can proceed independently behind the same transport interface once request/connection semantics stabilize. API/read-model work can also proceed in parallel. io_uring tuning starts after whole-engine correctness and a repeatable baseline.

Retain useful established features through a capability matrix: magnet/metainfo acquisition, v1/v2/hybrid identities, private-torrent behavior, tracker/DHT/peer exchange policy, transport/encryption/proxy modes actually supported, resume/recheck, selective download/padding, move/delete, and compatibility APIs. Each row names a production scenario and owner. An unsupported or broken capability is visible; it cannot quietly disappear during a throughput rewrite.

9. Dependency-ordered delivery gates

Gate	Shippable result	Exit evidence
G0 - Correctness and honest evidence	Regressions committed, urgent extent/rate/protocol defects repaired, valid baseline controller started	Multi-block v2 memory/spill tests; release-rate tests; request lifecycle fixtures; source/build identity; no unsupported performance claims
G1 - Ownership reference	Typed ranges/epochs, retained buffer owners, bounded operation/completion lifecycle, portable file/device service	Deterministic cancellation/late completion/short I/O tests; all live paths inventoried; accounting survives last-slice/kernel lifetime
G2 - V1 pipeline	Production v1 through shared ledger, fair grants, duplex I/O, completion-driven hash/disk path	Independent peers, no actor stall under delayed I/O, low-rate progress, bounded pressure, endgame and choke races
G3 - V2 and hybrid	Correct complete-piece handling, shared reservations, authenticated proof store, both hash views	Large/short/multi-block files, pure v2 interop, padding and nonzero offsets, large-file proofs, staged hybrid overlap tests
G4 - Durability and storage	Dirty-only fences, restart evidence, adaptive range cache, chunked recheck, scheduled copy	Crash matrix, no unrelated sync, canceled write accounting, copy depth invariance, unchanged secure filesystem invariants
G5 - Scale and transports	Real catalog/worker tests, deltas, event-driven reaping, bounded timers/control traffic, qualified uTP	Live 10k/100k catalog trials, mixed activity, partial/reordered/lossy transport tests, no task or memory leak
G6 - Hardware optimization	Continuous io_uring, proven cache/window/queue policies, same-source allocator experiments where warranted	Paired whole-client trials, compatible durability, helper ceilings, no bottleneck migration hidden by averages
G7 - Release qualification	Reproducible binaries, preserved capabilities, canary and rollback	Required matrix pass, recovery and bounded-resource soak, full result manifests, controlled release dwell and retained fallback

Some work within gates runs in parallel. Exit gates remain dependency-based: a fast NVMe microbenchmark cannot waive a failed v2 or crash test. `BACKLOG.json` encodes the cross-stream prerequisites; `BACKLOG.md` gives a readable index. Existing CI may run faster subsets on each PR, while hardware qualification runs on dedicated machines. Required failures remain failures instead of being relabeled flaky without diagnosis.

10. First implementation sequence

The first landing should include the reproduced tests and the extent correction with expanded geometry coverage. Keep the request-window regression red until its own fix lands; never hide it inside a broad test exclusion in CI. Repair the release-only webseed debit, aggregate torrent upload cap, explicit pacing wakeup, and advertised Fast lifecycle using dedicated regressions. These are small enough to review separately and important enough not to wait for the large refactor.

In parallel, build the deterministic core/operation harness and baseline observer. Introduce typed extents and retained buffer owners, then the shared ledger and portable service. Route one complete v1 transfer through the new core with existing APIs and resume format. Integrate one production vertical slice before spreading the new interfaces across every backend. Delete the obsolete live path only after the slice passes correctness and measured resource gates.

Assign one integration owner for transfer/storage/persistence contracts. Agents or engineers may own independent tickets, but no one should independently invent another pool, scheduler, file cache, or snapshot model. Each implementation PR states which owner changed, which invariant it preserves, what production route now uses it, what obsolete route was retired, and what exact evidence proves the result.

11. Effort and decision discipline

This is a substantial systems-engineering program. A credible planning range for a small experienced team is several months for a reliable replacement core and additional months of cross-platform, workload, and recovery qualification; six to twelve or more months is a plausible program envelope, not an estimate supported by measured team velocity. A solo implementation can take materially longer. Confidence in calendar estimates is low until G0/G1 expose integration cost.

Avoid committing to a giant unreviewable rewrite branch. Deliver vertical slices with explicit compatibility and measurable exits. Reject an optimization if it merely moves cost into unmeasured kernel memory, weakens integrity, reduces functionality, or makes a faster stage wait on a slower new bottleneck. Keep a good portable backend even if Linux eventually wins through a tuned specialized path.

Progress toward the goal must be demonstrated through correct content, bounded resources, deliberate ownership, proven scheduling, and reproducible wins on named workloads. The repository already contains useful pieces for that work. It does not contain evidence that a few configuration changes will achieve it.

Part III - Benchmark and release contract

Version 1.1 - proposed implementation specification, 2026-10-17

Status: build specification. Numeric targets are proposed gates, not achieved performance. **Confidence:** high that the current evidence needs these controls; moderate that the proposed first-stage budgets are achievable; unknown whether TorrentNG will beat each comparator until this suite runs.

The accompanying `benchmark-workloads.json` defines 28 scenarios, 92 explicit variants, six A1 API profiles, result/raw-sample/API-request schemas, counters, and validity rules. `benchmark-tickets.json` maps the work to implementation tickets. All three belong in the implementation agent's working context.

1. What this contract measures

The goal is the greatest useful, verified throughput within explicit CPU, memory, storage, latency, and correctness constraints. A low CPU percentage during a stalled download is not an efficiency result. A low heap number obtained by moving buffers into kernel cache is not a total-memory result. A benchmark that caps its receiver below the client's own upload ceiling cannot establish capacity.

Keep two experiments separate. **Capacity trials** remove artificial throughput limits and prove that helper peers, the network, and storage have sufficient capacity. **Matched-goodput trials** ask every client to deliver the same useful rate and compare CPU seconds, energy, memory, disk amplification, and latency. Failure to meet the setpoint fails that operating point; it does not earn a low-resource-use win.

Run two configuration tracks: fresh product defaults and equal external resources/features. In the second track, clients receive equal CPU, total-memory, network, storage, peer, protocol, encryption, and durability conditions. Internal queue sizes may differ within the same budgets. Record tuning procedures and effective settings rather than pretending identical internal settings are fair across architectures.

2. Evidence motivating the suite

The audit pinned TorrentNG commit `05d5c23886446f380552c73e0be480f03ae2d2ab`. Source inspection found inconsistent start/end clocks, synchronous sampling that lengthens the advertised cadence, a 100k router-memory test that consumes the router before sampling, and a hot-memory test that sums only ten synthetic rows. These are test-validity defects, not performance measurements.

Fresh audit probes supplied these allocation observations: the 1M-piece picker estimated 4,440,536 bytes while constructor allocation was 8,000,000 bytes; adding its first full peer incurred 11,618,752 further bytes and 166,665 allocation calls. A 100k registry snapshot invalidated by one changed row incurred 711,038 allocations and 40,746,421 gross bytes. Its single-run 83.6 ms wall time is diagnostic, not a stable latency estimate. These are allocator evidence, not RSS measurements. Existing 118 focused tests passed; three new v2 regressions failed the baseline. A narrow extent fix initially passed 27 focused v2 tests. Expanded boundary coverage then passed 30 focused tests, while the separate pipeline defect remained. Preserve these as identified regressions and controls; they do not establish full-client qualification.

3. Controller structure and adapter contract

Implement `perf/controller`, `perf/adapters`, `perf/observer`, `perf/swarm`, `perf/fixtures`, `perf/schema`, and `perf/analysis`. Store immutable results under `perf/results/<binary-sha>/<workload-hash>/<run-id>/`. The controller does not link TorrentNG internals into its correctness oracle.

Each client adapter implements:

1. `prepare`: create a fresh client-owned state root and report capability support.
2. `identify`: return binary hash, release/source identity, linked engine version, image digest, and executable command.
3. `configure` and `read_effective_settings`: apply settings and read back what the client actually accepted.
4. `arm`: add the fixture paused, establish allowed peers without payload exchange, and expose the zero-byte baseline.
5. `release`: start the common measurement barrier.
6. `status`: report completion, error, peer, and cumulative-byte fields with their semantics declared.
7. `quiesce`: stop new work and await outstanding operations.
8. `stop`: request normal shutdown, wait for process exit, and retain exit status and logs.

Unsupported features produce `unsupported`, adapter failures produce `fixture_error`, and valid but slow/broken clients produce `failed`. None becomes a numeric zero-throughput ranking. A comparator that fails startup needs an adapter/build repair before the comparison is complete.

4. Build and baseline identity

At run creation, resolve all source tags and container tags to immutable commit IDs/digests and persist them. Reject mutable references in qualification manifests. Record compiler version, target, release flags, features, allocator/tunables, lockfile hash, binary SHA-256, image digest, and linked libtorrent or other engine versions. Use a clean source tree for release evidence; dirty development experiments retain diff and untracked-content hashes.

Baselines are TorrentNG, qBittorrent, Transmission, Deluge, upstream rTorrent, and rqbit. qBittorrent and Deluge may share libtorrent-rasterbar; record that dependency so six products are not mistaken for six independent engines. rTorrent's separate libtorrent is a different project. Latest primary releases observed during this audit included rqbit 9.0.1, Transmission 4.1.3, and upstream rTorrent 0.16.25. The workload catalog retains the repository's existing image digests as historical references; it does not assume they remain the desired baseline forever.

Prebuild every binary before trials. Pin independent tools at run time too: fixture generator, tracker, peer simulator, `iperf3`, `fio`, hash implementation, profiler, and result parser. Save `--version` output and executable hashes. A later source commit cannot inherit qualification merely because a script found an older PASS log.

5. State machine, timestamps, and denominators

Every run transitions through `created -> prepared -> fixture_verified -> armed -> measuring -> transfer_complete -> quiesced -> externally_verified -> finished`, or a declared terminal failure. Record every transition as an append-only event with monotonic nanoseconds, UTC timestamp, actor, reason, and relevant cumulative byte counters. Save the mapping between monotonic and UTC clocks; never calculate elapsed transfer time from wall-clock dates.

`t0` is the release barrier after all peers are armed and baseline payload bytes are zero. For startup scenarios, the catalog explicitly moves `t0` before metadata loading. `t_verified` is the first dedicated completion observation that all required content has passed the receiving client's checks. The adapter declares its observation method and uncertainty. Completion observation runs independently of the resource sampler; polling cadence must be <=50 ms for short trials. Save the upper-bound observation delay.

The denominator is **unique requested valid payload bytes delivered to the intended recipient**. Torrent padding, protocol overhead, retransmissions, failed hashes, discarded data, and repeated/endgame blocks are separate counters. For multi-leecher seeding, count unique bytes separately for each intended recipient, then sum; a repeated block to the same recipient counts once. Selective download uses the committed selected-file/range manifest. Partial completion never qualifies as successful full-object goodput.

Useful goodput is verified useful bytes divided by `t_verified - t0`. Record metadata/setup time, time to first verified piece, transfer time, flush time, external hash time, and shutdown time independently. Check every complete object with an independent full hash outside the transfer timer.

Durability has two explicit tracks. `client_default` records the client's ordinary behavior and makes no inferred fsync guarantee. `uniform_fsync_boundary` quiesces and stops the client, then the harness fsyncs all benchmark-owned payload files and required directory metadata; its separate `t_durable` includes that additional persistence work and shutdown. Report harness flush CPU separately and include it in whole-system cost. Do not attribute harness-added durability to the client. Crash/recovery tests compare outcomes against the declared engine persistence contract.

6. Resource accounting

Use a dedicated observer and client cgroup. Read cumulative `cpu.stat` usage/user/system microseconds at the start and end; do not reconstruct CPU time by summing only surviving threads or averaging `docker stats` percentages. CPU efficiency is CPU seconds per verified GiB. Record whole-host isolated CPU as a separate quantity because networking IRQ/softirq work can occur outside the client cgroup.

Collect timestamped `memory.current`, `memory.peak`, memory events, anonymous/file/kernel/socket categories, pressure stalls, `io.stat`, and process RSS/PSS. Preserve cumulative counters and reset/wrap information. Do not sum overlapping memory subcategories. The complete charged footprint, private/anonymous memory, reclaimable cache, and logical engine ownership are separate views. Required missing counters invalidate their gate; zero is a real value, never a sentinel for unavailable data.

Sample lightweight counters every 100-250 ms for short runs and one second for long runs, using deadlines `t0 + n x interval`; log missed deadlines. Expensive smaps, allocator, syscall, and stack profiles run in separate diagnostic trials. Validate profiler overhead with paired on/off runs. Metrics collection must not instantiate storage runtimes, cache sweepers, rings, or worker threads.

Where real power instrumentation exists, report gross and idle-baseline-adjusted joules per verified GiB, exact energy domains, counter wraps, and measurement interval. Package energy excludes other hardware unless measured separately. If energy counters or wall power are unavailable, energy is `null` with a reason. Never infer joules from CPU percent.

7. Filesystem, cache, and hardware controls

Create all data beneath a new benchmark-owned root containing an ownership manifest. Never clean a root supplied without that identity. Cleanup resolves paths, rejects symlink escapes, and deletes only IDs created by this run. No shared client data, live torrents, global cache drops, or destructive device preparation belongs in this harness.

Use independent inodes and incompressible deterministic data for cold comparisons. Hash preparation warms data, so establish cache state afterward. For cold cases, flush dirty data, perform permitted scoped eviction, record the return value, and verify residency/device reads. If cold state cannot be established, invalidate that cold case or label a separate warm case. `POSIX_FADV_DONTNEED` is a request, not proof. Large sustained cases use a working set at least four times the effective cache budget.

Record CPU topology/frequency/NUMA, RAM/swap, cgroup controls, kernel, storage model/firmware, filesystem/mount options, physical-device topology, network interface/MTU/offloads, and thermal state. Separate HDD, SATA SSD, NVMe, and network filesystem profiles. NFS/SMB tests include server cache/disk telemetry; zero client block reads do not mean zero storage work. Prefer a separate helper host for high-rate trials; otherwise isolate helper CPU and disks and demonstrate headroom.

Disk amplification has two reported numerators: payload-file-attributed physical bytes and whole-device bytes. Report journal/metadata/preallocation separately. Useful bytes are the same verified denominator. Tiny-file metadata cost cannot be compared to a large-file write envelope without its own workload label.

8. Independent ceilings and protocol controls

Before engine comparisons, establish:

- `iperf3` TCP/UDP payload capacity for the configured latency/loss/MTU path.
- A standalone plain-TCP sender/receiver with bounded buffers and independent content hashing.
- Hash-only SHA-1/SHA-256/hybrid capacity with the same CPU budget and buffer sizes.
- `fio` or equivalent direct/buffered read/write envelopes at relevant block sizes, queue depths, and mixed workloads.
- A plain TCP -> hash -> disk and disk -> TCP pipeline with the same verification and persistence boundaries.

These controls locate ceilings; a sequential disk test does not predict a random multifile torrent workload. The independent pipeline must not import TorrentNG's scheduler/picker/runtime or silently reuse the mechanism under test. Record helper saturation and observer overhead so they cannot masquerade as a client limit.

Network fixtures use fixed peer availability, rates, choke schedules, loss, jitter, and disconnect seeds. Test pure TCP, pure uTP where supported, and mixed transport independently. Include one fast peer, many peers, constrained advertised request queues, slow tricklers, and changing piece availability. Capture actual wire behavior for disputed counters.

Theoretical BDP is bytes/second x service delay. At 1 Gbit/s and 80 ms, network BDP alone is 10,000,000 bytes, approximately 611 requests of 16 KiB. This is not permission to send 611 requests to every peer: use per-peer rate share, negotiated limits, disk service delay, and a global admitted byte budget. Test whether the adaptive window sustains bandwidth without unbounded memory.

`N1/lan_10g` is the dedicated 10Gb/s LAN gate: one peer, 32 GiB payload, 16 GiB client memory limit, and a separate helper with at least 64 GiB RAM and verified source residency. Variant scalar settings

override fixture defaults; explicit variant target lists replace scenario-level lists. WAN variants cannot inherit the LAN capacity target.

API load and pressure profiles

A1 uses the versioned `a1-ui-mix-v1` fixture and manifest v1.1.0. Normal profiles offer 0, 5, 100, or 400 aggregate HTTP requests/second through 0, 1, 16, or 64 client sessions. The fixed mix is 50% bounded catalog pages, 20% torrent details, 20% session transfer information, 5% pause, and 5% resume. Mutations address eight isolated control torrents with no peers; the 32 payload torrents continue at the declared 100 MiB/s aggregate offered rate. Control overhead remains in client resource costs. API traffic uses a management path outside the payload-rate shaper, with independent generator headroom.

Expand and hash the seeded open-loop request schedule before release. Record each scheduled arrival, send, response, rejection, timeout, and capacity-limited submission. Latency begins at scheduled arrival; waiting in the generator cannot disappear. Normal control p99 passes only when every scheduled control is correctly accepted. Endpoint mappings, operation semantics, connection bounds, and control fixtures are pinned; missing equivalent capability is an unsupported variant.

The overload profile offers 100 requests/s for seconds 0-20, 8,000 for 20-50, then 100 through completion. A separate profile offers 100 requests/s while 16 SSE consumers stop reading during seconds 20-80. Both require bounded queues, explicit outcomes, continuing useful transfer, and normal service recovery within ten seconds after pressure ends. Normal p99 does not apply inside the intentional pressure interval. If the fixed high rate never induces backpressure, report that rejection was not exercised. SSE gaps require the protocol's valid resynchronization or explicit disconnection. Ordinary profiles consume one event per consumer per 250 ms. Metrics-overhead comparisons preserve the same API profile; a busier UI is a different workload.

9. Trials, statistics, and invalidity

Use at least five randomized paired trials during development and ten for initial release qualification, increasing trials only when the predeclared confidence interval is too wide. Pair the same workload/cache/resource state and balance client order. Save the random seed and complete planned order before execution. Competitors do not run simultaneously against the same disk/network bottleneck.

Report each raw result, median, dispersion, paired ratios, and bootstrap 95% confidence intervals. Latency reports include p50/p95/p99 and maximum. Do not discard a valid slow run. Record timeouts and integrity failures as failures, with partial progress shown separately. Only predeclared infrastructure conditions invalidate a trial: wrong identity/settings, missing counters, nonzero prestart bytes, observer timing failure, helper saturation, uncontrolled cache state, unrelated host activity beyond its calibrated bound, or an actual fixture error. Save the evidence and rerun the entire pair.

Content corruption, a client crash, missed rate target, resource-budget breach, or an engine-created stall is a valid failed trial - not infrastructure invalidity. Public swarms are a separate compatibility sanity track; changing peers prevent them from deciding the controlled engine ranking.

10. Proposed gates

The workload catalog contains precise fixture-specific proposed envelopes. Initial examples are: quiet Linux daemon <=32 MiB anonymous and <=64 MiB PSS; 100k bounded dormant catalog <=256 MiB anonymous and <=512 FDs while still live; 1k hot torrents/four peers each at the prescribed aggregate load <=512 MiB anonymous and <=2 GiB total charged memory. These are goals requiring measurement, not product promises.

For basic 1Gb TCP large-file LAN cases, propose >=105 MiB/s and >=90% of the measured payload network ceiling. For provisioned 10Gb NVMe, propose >=900 MiB/s with the same calibrated-ceiling condition. On each mandatory comparable case, initially target >=95% of the fastest valid comparator's useful goodput, with no correctness failures. At matched goodput, target materially lower CPU/GiB rather than a smaller percentage while doing less work.

A leadership claim requires a material, statistically supported win in the named workload/metric; proposed threshold is >=10% improvement with a paired 95% interval excluding parity. If all clients saturate the link, claim resource savings at that speed. Publish bad cases and failures alongside wins. All numbers may be revised through a recorded design decision before new experiments; never change a definition after observing a bad result to manufacture PASS.

11. Qualification and rollout

First repair the observer, timing contract, live-object sampling, metric side effects, and evidence schema. Validate the harness against known CPU/memory/I/O programs on an actual host with reliable counters; this audit's tool environment exposed inconsistent `/proc` process identity and cannot supply those measurements.

Freeze and measure the audited baseline plus working references. Then each refactor runs production-engine invariant tests and a same-workload previous/new comparison. Host correctness CI remains fast. Dedicated stable hardware handles performance regressions, and changed data-path components trigger its relevant matrix.

Candidates require the core controlled workload matrix, 72-hour mixed load, crash/restart and storage-error coverage, and a representative seven-day retention soak before broad stability claims. Canary rollout uses an isolated copy/snapshot of explicitly chosen workload state, retains the previous binary and recoverable state, and stops promotion on correctness, resource, or throughput regressions. Qualification attaches to the exact binary/source/workload manifest. Historical PASS text is never current-artifact proof.

Primary references

- libtorrent 2.0 disk I/O migration: mmap/kernel-cache changes and independent hash/I/O controls.
- libtorrent tuning: buffering/scalability tradeoffs; its older disk-thread subsection explicitly identifies itself as outdated.
- Linux cgroup v2: cumulative CPU, memory categories/ownership, I/O and pressure counters.
- rqbit releases, Transmission releases, and rTorrent releases: observed reference releases; resolve immutable identities again at run creation.

Part IV - Proposed acceptance scorecard

Status: proposed, not measured or achieved. These gates select the most useful targets from `benchmark-workloads.json`; the full benchmark contract controls their interpretation. They describe named Linux workloads, resource envelopes, and service levels. They promise no universal speed, memory ceiling, or superiority over every client. Freeze each fixture, target, and measurement definition before collecting qualification results; record later changes as new experiment versions.

Conditions for a qualifying result

Pin the source tree, lockfile, executable/image hashes, actual embedded libraries, effective settings, fixture, hardware, kernel, filesystem, helper, and observer. Retain raw timestamps, cumulative counters, peaks, failures, and independent integrity checks. Use ten randomized paired release trials with 95% intervals. Missing required evidence invalidates the pair; client corruption, timeout, crash, or budget breach is a failure. Retain the live engine through resource sampling. Memory categories below are separate measurements, not quantities to add together.

Initial gates

Gate	Exact workload qualification	Proposed acceptance
Integrity and recovery	Every applicable production workload, including v2 tails/multi-block pieces and declared R1/R2 fault scenarios	**Zero** unexpected hash mismatches, false completions, or violations of declared recovery semantics. Expected injected failures must remain visible.
LAN capacity	`N1/lan_1g`: 8 GiB; `N1/lan_10g`: 32 GiB with 16 GiB client limit and separate >=64 GiB helper. Both: x64 NVMe, one TCP peer, 0.2 ms RTT, verified warm source and calibrated helper headroom	Respectively **>=105** and **>=900 MiB/s** verified useful goodput, each **>=90%** of its calibrated payload ceiling. WAN variants have separate gates.
Empty idle	`M1/metrics-off` and `M1/metrics-on`, separately: named x64 Linux profile, no torrents, 600 seconds after initialization	Peak anonymous memory **<=32 MiB**, peak process PSS **<=64 MiB**, mean CPU **<=0.5% of one core**, peak OS threads **<=32**.
Dormant catalog	`M2/catalog-100000`: production restore, metadata on disk/lazily resident, names <=80 bytes, one configured tracker per torrent, no peers; live owner retained for 600 seconds	Peak anonymous memory **<=256 MiB**; incremental owned allocation **<=2 KiB/torrent**; peak descriptors **<=512**. Retain startup/restore peaks separately.
Active catalog	`M3/1000-hot`: x64 NVMe, 1,000 active torrents, four peers each, 16 GiB total payload, 100 MiB/s aggregate offered load, 64 MiB payload credits, 256 MiB cache cap	Verified useful rate **>=98 MiB/s**; peak anonymous memory **<=512 MiB**, total charged memory **<=2 GiB**, descriptors **<=4,640**. Aggregate every owner.
Download writes	`S1/1-torrents`: one 64 GiB file, ext4 HDD, 16 peers, 512 MiB memory limit, declared uniform fsync boundary	Attributed payload physical writes/useful bytes **<=1.15**; whole-device writes/useful bytes **<=1.30**. Include the declared flush window.

Gate	Exact workload qualification	Proposed acceptance
Random seed reads	`S2/128-leechers`: HDD, 512 MiB limit, 32-block request windows, random full-piece subsets across 64 GiB, 32 GiB total recipient demand	Attributed payload physical reads/unique verified recipient bytes **<=1.25**. Report competing readahead policies separately.
Control responsiveness	`A1/api-16`: 32 active torrents, 100 MiB/s offered load, 16 API clients, four SSE consumers, versioned `a1-ui-mix-v1` at **100 aggregate requests/s**	Scheduled-arrival-to-acceptance **p99 <=100 ms**, every scheduled control correctly accepted, useful transfer **>=98 MiB/s**. Physical completion is separate.
Overload and stalled consumers	`A1/api-overload-8000`: 8,000 requests/s during seconds 20-50; `A1/sse-stalled-60s`: 16 consumers stop reading during seconds 20-80; otherwise 100 requests/s	API/SSE pending bytes **<=8/4 MiB**, explicit terminal outcomes **<=1 second**, normal service recovers **<=10 seconds** after pressure ends. No normal-p99 requirement during pressure.
Retention	`R3/72h-candidate` and `R3/7d-release`: the pinned mixed seed/download/churn trace after warmup	Last-quarter median anonymous bytes **<=first steady-state-quarter median + max(16 MiB, 5%)**; no per-cycle descriptor/task growth. Integrity and queue bounds still apply.

Comparative claims

Within identical resources, cache state, capability, and durability, propose at least **95%** of the fastest valid comparator's useful goodput. At matched load, require **98%** of the offered rate before comparing cumulative CPU seconds or measured joules per verified GiB. A leadership claim needs a named metric with **>=10%** improvement and a paired 95% interval excluding parity. Metrics on/off must preserve service topology and change goodput/CPU-per-GiB by **<=2%**.

API mix, endpoint mappings, arrival traces, and versioned fixture hashes are fixed in the catalog and run manifest. Uninstrumented energy remains unavailable, never zero.

Part V - Implementation work index

63 work packages: 15 integration/control, 16 storage, 16 transfer, and 16 benchmark. This is proposed work, not a completion log. `BACKLOG.json` contains scope, paths, dependencies, acceptance and tests for every ticket.

The table follows a topological order. Tickets in the same ready wave can start together when separate owners/resources permit. A wave is a dependency frontier, not a calendar sprint. Cross-stream prerequisites in the combined JSON take precedence over local stream views.

Ticket	Priority	Work package	Prerequisites
SYS-001	P0	Pin the reference, capability matrix and shared ownership contracts	None
IO-001	P0	Define storage evidence contracts and deterministic fault harness	SYS-001
NET-101	P0	Define typed transfer coordinates and protocol geometry adapters	SYS-001
PERF-101	P0	Commit measurement contract and strict result schemas	SYS-001
SYS-002	P0	Land v2 reproductions and a reviewed extent correction	SYS-001
SYS-003	P0	Repair immediate release-rate and negotiated-capability behavior	SYS-001
SYS-004	P0	Introduce compact identities and honest whole-owner metadata accounting	SYS-001
SYS-009	P0	Exercise the production database worker and coalesce progress	SYS-001
SYS-011	P0	Make live observability passive and owner-specific	SYS-001
IO-002	P0	Separate block and piece extents and repair multi-block v2 transfers	IO-001, SYS-002, NET-101
IO-003	P0	Carry capacity leases through immutable payload ownership	IO-001
IO-004	P0	Unify the secure file catalog and descriptor lifecycle	IO-001
NET-102	P0	Build a deterministic transfer simulator and invariant oracle	NET-101
PERF-102	P0	Build independent cumulative resource observer	PERF-101
PERF-103	P0	Implement client adapters with pinned identity and settings readback	PERF-101
PERF-105	P0	Build scoped deterministic fixtures and cache controls	PERF-101

Ticket	Priority	Work package	Prerequisites
SYS-005	P1	Publish registry chunks and revision deltas without global deep clones	SYS-004
SYS-007	P1	Replace per-command task scans with supervised completions	SYS-004
SYS-008	P1	Replace quadratic timer cancellation and stagger bulk work	SYS-004
SYS-012	P0	Preserve feature and platform compatibility through production fixtures	SYS-001, SYS-002, SYS-003
IO-005	P0	Make admission and completion ownership bounded and cancellation-safe	IO-003, IO-004
NET-103	P0	Reserve payload capacity before requests and carry leased buffer spans	NET-101, NET-102, IO-003
NET-112	P1	Select a uTP backend using a reproducible integration evaluation	NET-101, NET-102
PERF-104	P0	Implement common timing barriers and useful-byte accounting	PERF-101, PERF-102, PERF-103
PERF-106	P0	Create deterministic independent swarm and network controls	PERF-101, PERF-105
PERF-108	P0	Remove metrics-created shadow storage runtime and stabilize accounting	PERF-101, PERF-102, SYS-011
SYS-006	P1	Route APIs through bounded delta, page and sort projections	SYS-005
SYS-010	P1	Bound discovery and control I/O and decouple scrape from announce	SYS-004, SYS-008
IO-006	P0	Build one device service over the portable reference backend	IO-005
IO-007	P0	Create batched hash jobs and shared hybrid content views	IO-002, IO-003, IO-005
NET-104	P0	Implement the authoritative request and content ownership ledger	NET-101, NET-102, NET-103, IO-005
PERF-107	P0	Implement independent network hash disk and pipeline ceilings	PERF-102, PERF-105, PERF-106
PERF-109	P0	Replace synthetic capacity proxies with live production fixtures	PERF-102, PERF-103, PERF-104, PERF-105, PERF-108
PERF-112	P0	Enforce v2 and persistence invariants in full transfer paths	PERF-104, PERF-105, PERF-106
IO-012	P1	Stream recheck through large chunks and persistent sparse cursors	IO-006, IO-007

Ticket	Priority	Work package	Prerequisites
NET-105	P0	Implement negotiated peer protocol request semantics	NET-104
NET-109	P1	Rebuild piece selection around active candidates and accurate accounting	NET-104
PERF-110	P0	Automate 28-case catalog and randomized paired execution	PERF-103, PERF-104, PERF-105, PERF-106, PERF-107, PERF-109
PERF-111	P1	Add picker timer and changed-snapshot allocation regression probes	PERF-101, PERF-102, PERF-109
SYS-014	P1	Qualify complete catalog cost and mixed-load responsiveness	SYS-005, SYS-006, SYS-007, SYS-008, SYS-009, SYS-010, PERF-109
NET-106	P1	Separate peer read progress from write and storage waits	NET-103, NET-104, NET-105
PERF-113	P1	Implement statistical analysis and explicit scorecard	PERF-101, PERF-102, PERF-104, PERF-110
IO-008	P0	Move transfer actors to completion-driven storage and hashing	IO-002, IO-006, IO-007, NET-104, NET-106
NET-107	P1	Replace replicated token buckets with hierarchical fair rate scheduling	NET-103, NET-104, NET-106
NET-113	P1	Deliver the selected windowed uTP transport behind the shared runtime	NET-103, NET-106, NET-112
PERF-114	P0	Replace text-grep qualification with exact artifact manifests	PERF-101, PERF-113
IO-009	P0	Implement dirty-only generation fences and grouped barriers	IO-004, IO-006, IO-008
IO-011	P1	Replace unconditional readahead with useful range caching	IO-004, IO-006, IO-008
NET-108	P1	Implement adaptive request windows and deadline-driven refill	NET-104, NET-106, NET-107
NET-110	P1	Implement bounded shared upload service and eliminate valid-demand drops	NET-103, NET-105, NET-106, NET-107
PERF-115	P1	Integrate hosted correctness and dedicated hardware regression lanes	PERF-110, PERF-111, PERF-112, PERF-113, PERF-114
IO-010	P0	Bind resume persistence and crash recovery to barrier evidence	IO-009, SYS-009
IO-014	P1	Schedule secure copies and verify each leaf once	IO-006, IO-009, IO-012
IO-015	P1	Add continuous io_uring with retained safe registrations	IO-006, IO-009

Ticket	Priority	Work package	Prerequisites
NET-111	P1	Implement fair seed and download choking policies	NET-104, NET-107, NET-110
IO-013	P1	Persist authenticated v2 Merkle nodes and serve bounded proofs	IO-007, IO-010, IO-012
NET-114	P1	Move v1, pure-v2 and hybrid transfer execution onto the shared core	NET-104, NET-105, NET-108, NET-109, NET-110, IO-008, IO-010
NET-115	P1	Unify source scheduling and batch progress publication	NET-106, NET-107, NET-108, NET-114, SYS-009
SYS-013	P0	Integrate one-writer migration and durable rollback	NET-114, IO-010, SYS-005, SYS-007, SYS-009, SYS-012
IO-016	P0	Gate rollout on live-engine parity, recovery and hardware workloads	IO-008, IO-009, IO-010, IO-011, IO-012, IO-013, IO-014, IO-015, PERF-110, SYS-013
NET-116	P1	Gate capability rollout with independent correctness and performance evidence	NET-105, NET-108, NET-109, NET-111, NET-113, NET-114, NET-115, PERF-112, PERF-113, SYS-013
PERF-116	P1	Run candidate soak recovery and reversible canary qualification	PERF-112, PERF-114, PERF-115, SYS-013, SYS-014
SYS-015	P0	Integrate final gates, independent review and reproducible release	SYS-013, SYS-014, NET-116, IO-016, PERF-116

Execution frontiers

- Wave 1: SYS-001
- Wave 2: IO-001, NET-101, PERF-101, SYS-002, SYS-003, SYS-004, SYS-009, SYS-011
- Wave 3: IO-002, IO-003, IO-004, NET-102, PERF-102, PERF-103, PERF-105, SYS-005, SYS-007, SYS-008, SYS-012
- Wave 4: IO-005, NET-103, NET-112, PERF-104, PERF-106, PERF-108, SYS-006, SYS-010
- Wave 5: IO-006, IO-007, NET-104, PERF-107, PERF-109, PERF-112
- Wave 6: IO-012, NET-105, NET-109, PERF-110, PERF-111, SYS-014
- Wave 7: NET-106, PERF-113
- Wave 8: IO-008, NET-107, NET-113, PERF-114
- Wave 9: IO-009, IO-011, NET-108, NET-110, PERF-115
- Wave 10: IO-010, IO-014, IO-015, NET-111
- Wave 11: IO-013, NET-114
- Wave 12: NET-115, SYS-013
- Wave 13: IO-016, NET-116, PERF-116
- Wave 14: SYS-015

Source index

All repository entries below refer to commit 05d5c23886446f380552c73e0be480f03ae2d2ab. In-report blue source labels and this index are clickable. Primary external references are listed separately; they support protocol and design constraints, not claimed TorrentNG speedups.

Report reference	Pinned repository path and lines
v2-handler	crates/rt-engine/src/torrent_task_v2.rs : 3402-3543
v2-map	crates/rt-piece-map/src/v2.rs : 86-150
v2-fixtures	crates/rt-engine/src/torrent_task_v2.rs : 4229-4278
v2-fill	crates/rt-engine/src/torrent_task_v2.rs : 3311-3398
v2-peer	crates/rt-engine/src/torrent_task_v2.rs : 3013-3309
v1-dispatch	crates/rt-engine/src/torrent_task.rs : 2569-2625
v1-receive	crates/rt-engine/src/torrent_task/peer_transfer.rs : 315-479
v1-window	crates/rt-engine/src/torrent_task.rs : 795-1041
upload-guard	crates/rt-engine/src/torrent_task.rs : 102-116
upload-loop	crates/rt-engine/src/torrent_task.rs : 7181-7257
utp-send	crates/rt-utp/src/transport.rs : 616-708
utp-state	crates/rt-utp/src/state.rs : 330-378
fast-handshake	crates/rt-engine/src/torrent_task.rs : 6125-6150
peer-commands	crates/rt-engine/src/torrent_task.rs : 1504-1518
peer-session-upload	crates/rt-engine/src/torrent_task/peer_session.rs : 243-275
peer-rate	crates/rt-engine/src/torrent_task.rs : 7418-7455
webseed-tokens	crates/rt-engine/src/torrent_task/peer_session.rs : 639-679
peer-refill	crates/rt-engine/src/torrent_task/peer_transfer.rs : 257-307
choker	crates/rt-peer-manager/src/choker.rs : 53-103
rate-decay	crates/rt-engine/src/torrent_task.rs : 5702-5734
assembly-budget	crates/rt-engine/src/torrent_task/peer_transfer.rs : 483-561
picker-memory	crates/rt-piece-picker/src/picker.rs : 153-213
availability	crates/rt-piece-picker/src/availability.rs : 1-112
picker-select	crates/rt-piece-picker/src/picker.rs : 348-448
frame-owner	crates/rt-storage/src/frame.rs : 241-308
uring-buffer	crates/rt-storage/src/backend.rs : 879-916
v1-copies	crates/rt-engine/src/torrent_task.rs : 4406-4523
upload-copy	crates/rt-engine/src/torrent_task.rs : 7639-7680
file-pool	crates/rt-storage/src/scheduler.rs : 467-617
path-identity	crates/rt-storage/src/scheduler.rs : 400-430
read-validation	crates/rt-storage/src/scheduler.rs : 2890-2914

Report reference	Pinned repository path and lines
read-expansion	crates/rt-storage/src/scheduler.rs : 2035-2132
range-cache	crates/rt-storage/src/scheduler.rs : 2212-2271
sync-all	crates/rt-storage/src/scheduler.rs : 2477-2558
shared-storage	crates/rt-storage/src/scheduler.rs : 801-894
uring-loop	crates/rt-storage/src/backend.rs : 827-857
uring-registration	crates/rt-storage/src/backend.rs : 994-1147
write-lifetime	crates/rt-storage/src/scheduler.rs : 2285-2348
write-watcher	crates/rt-storage/src/scheduler.rs : 1157-1175
generic-elevator	crates/rt-storage/src/elevator.rs : 1-180
move-job	crates/rt-engine/src/storage_jobs.rs : 1500-1534
v2-recheck	crates/rt-storage/src/verify.rs : 331-414
v2-proofs	crates/rt-engine/src/torrent_task_v2.rs : 3683-3925
secure-copy	crates/rt-storage/src/secure_fs.rs : 479-571
secure-verify	crates/rt-storage/src/secure_fs.rs : 1040-1110
registry-snapshot	crates/rt-session/src/registry.rs : 595-634
native-snapshot	crates/rt-api-native/src/state.rs : 340-400
transmission-snapshot	crates/rt-api-transmission/src/lib.rs : 1995-2020
dormant-estimate	crates/rt-engine/src/tier.rs : 192-253
timer-wheel	crates/rt-engine/src/tier.rs : 497-595
engine-dispatch	crates/rt-engine/src/engine.rs : 2350-2390
reap-helper	crates/rt-engine/src/engine.rs : 1913-1921
reap-use	crates/rt-engine/src/engine/lifecycle.rs : 1535-1570
metrics-runtime	crates/rt-api-native/src/handlers.rs : 8357-8387
runtime-init	crates/rt-storage/src/runtime.rs : 44-112
db-worker	crates/rt-engine/src/db_worker.rs : 235-302
engine-db	crates/rt-engine/src/engine.rs : 1923-1956
scale-router	crates/rt-metrics/tests/scale.rs : 113-241
scale-top	crates/rt-engine/src/command.rs : 2140-2171
benchmark-caps	scripts/client_performance_benchmark.py : 44-52
benchmark-seed	scripts/client_performance_benchmark.py : 1635-1684
benchmark-download	scripts/client_performance_benchmark.py : 1749-1761
benchmark-sampling	scripts/client_performance_benchmark.py : 1294-1394
cpu-sampler	scripts/backend_engine_benchmark.sh : 254-273
seed-cache	scripts/client_performance_benchmark.py : 1790-1808
storage-microbench	crates/rt-storage/tests/storage_real_device.rs : 118-521

Report reference	Pinned repository path and lines
historical13	certification/reports/client-performance-benchmark-20261013.md : 3-78
historical14	certification/reports/native-memory-optimization-20261014-agentvm.md : 10-71
certification	scripts/memory_roadmap_certification.sh : 11-157
tracker-announce	crates/rt-engine/src/tracker_runtime.rs : 233-258
tracker-scrape	crates/rt-engine/src/tracker_runtime.rs : 490-515
http-timeout	crates/rt-config/src/lib.rs : 601-614

Primary technical references

bep3: https://www.bittorrent.org/beps/bep_0003.html

bep6: https://www.bittorrent.org/beps/bep_0006.html

bep10: https://www.bittorrent.org/beps/bep_0010.html

bep29: https://www.bittorrent.org/beps/bep_0029.html

bep52: https://www.bittorrent.org/beps/bep_0052.html

bytes-owner: https://docs.rs/bytes/1.11.1/bytes/struct.Bytes.html#method.from_owner

uring-manual: https://man7.org/linux/man-pages/man2/io_uring_enter.2.html

cgroup: https://docs.kernel.org/admin-guide/cgroup-v2.html

libtorrent-upgrade: https://libtorrent.org/upgrade_to_2.0-ref.html

libtorrent-tuning: https://www.libtorrent.org/tuning-ref.html

rqbit: https://github.com/ikatson/rqbit/releases

tokio-blocking: https://docs.rs/tokio/latest/tokio/task/fn.spawn_blocking.html